Part A: Conceptual Foundation (Theory)

1. What are Ensemble Learning Methods and why are they effective?

Ensemble Learning is a machine learning technique that combines multiple models to make better predictions. It improves accuracy and reduces errors compared to using a single model.

2. Explain the difference between Bagging, Boosting, and Voting.

- Bagging: Trains multiple models independently on different samples of data and combines their predictions. Example: Random Forest.
- Boosting: Trains models one by one, where each new model focuses on correcting previous errors. Example: AdaBoost, XGBoost.
- Voting: Combines predictions from multiple models and selects the final output using majority voting or average probability. Example: Voting Classifier.

3. What is Bias-Variance Trade-off, and how do ensemble methods address it?

Bias is the error caused by a model being too simple. Variance is the error caused by a model being too sensitive to training data. Ensemble methods combine multiple models to balance bias and variance and improve accuracy.

4. Explain Voting Classifier and Stacking Ensemble.

- Voting Classifier: Combines predictions from multiple models and gives the final result using majority voting (Hard Voting) or average probabilities (Soft Voting).
- Stacking Ensemble: Combines multiple models and uses another model called a Meta-model to make the final prediction.

5. Compare AdaBoost, Gradient Boosting, LightGBM, and XGBoost conceptually.

- AdaBoost: Focuses more on incorrectly predicted data points by giving them higher weights.
- Gradient Boosting: Builds models one by one, where each new model corrects previous errors using gradients.
- LightGBM: A fast and memory-efficient boosting algorithm that uses leaf-wise tree growth.
- XGBoost: An optimized boosting algorithm that uses regularization to reduce overfitting and improve performance.

All four are boosting algorithms, but they differ in training speed, tree-building methods, and optimization techniques.

Part B: Dataset Understanding & Preparation

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:
```
df = pd.read_csv(r"C:\Users\Ibhrahim ansari\Desktop\final project\Smart_Outcome_Predictor_Dataset_5200 - Smart_Outcome_Predictor_Dataset_5200.csv.csv")
df.head()
```

[2]:

	student_id	age	country_region	device_type	education_background	course_level	course_category	course_start_date	week_of_year	sessions	time_spent_hours	vi
0	700001	32	Europe	Laptop	Undergrad	Intermediate	Business	2024-03-18	12	1	7.6	
1	700002	17	Europe	Laptop	Undergrad	Intermediate	Programming	2024-08-22	34	16	27.2	
2	700003	25	Europe	Mobile	Graduate	Advanced	Programming	2024-09-28	39	6	7.1	
3	700004	26	Asia	Mobile	Undergrad	Beginner	Design	2024-03-09	10	34	22.1	
4	700005	26	Asia	Tablet	WorkingPro	Advanced	Business	2024-03-21	12	22	32.3	

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5200 entries, 0 to 5199
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   student_id            5200 non-null   int64
 1   age                   5200 non-null   int64
 2   country_region        5200 non-null   object
 3   device_type           5200 non-null   object
 4   education_background  5200 non-null   object
 5   course_level          5200 non-null   object
 6   course_category       5200 non-null   object
 7   course_start_date     5200 non-null   object
 8   week_of_year          5200 non-null   int64
 9   sessions              5200 non-null   int64
 10  time_spent_hours      5088 non-null   float64
 11  videos_watched        5200 non-null   int64
```

[4]:
```
print("Dataset Shape:",df.shape)
```

```
Dataset Shape: (5200, 19)
```

[5]:
```
df.describe()
```

[5]:

	student_id	age	week_of_year	sessions	time_spent_hours	videos_watched	quiz_attempts	assignments_submitted	forum_posts	avg_quiz_score
count	5200.000000	5200.000000	5200.000000	5200.000000	5088.000000	5200.000000	5200.000000	5200.000000	5200.000000	5119.000000
mean	702600.500000	24.339615	26.621538	19.121923	18.216156	25.852885	6.995769	3.835769	1.585577	66.842626
std	1501.255031	5.578864	15.007731	10.726708	10.734114	17.720493	2.568017	1.951802	1.270640	13.813992
min	700001.000000	16.000000	1.000000	1.000000	0.600000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	701300.750000	20.000000	14.000000	11.000000	10.300000	13.000000	5.000000	2.000000	1.000000	57.500000
50%	702600.500000	24.000000	27.000000	17.000000	16.100000	23.000000	7.000000	4.000000	1.000000	66.900000
75%	703900.250000	28.000000	40.000000	25.000000	23.800000	35.000000	9.000000	5.000000	2.000000	76.200000
max	705200.000000	45.000000	52.000000	94.000000	76.800000	158.000000	20.000000	12.000000	8.000000	100.000000

[6]:
```
missing_summary = df.isna().sum()[df.isna().sum()>0]
print("Missing values:\n",missing_summary)
```

```
Missing values:
 time_spent_hours    112
avg_quiz_score       81
attendance_rate      80
dtype: int64
```

[7]:
```
dup_val = df.duplicated().sum()
dup_val
```

[7]:
```
np.int64(0)
```

[8]:
```
from sklearn.impute import SimpleImputer
si = SimpleImputer(strategy="median")

df[["time_spent_hours", "avg_quiz_score", "attendance_rate"]] = si.fit_transform(df[["time_spent_hours", "avg_quiz_score", "attendance_rate"]])
```

[9]:
```
df["course_start_date"] = pd.to_datetime(df["course_start_date"])
```

[10]:
```
df.drop(columns=["student_id", "course_start_date"], inplace=True)
```

[11]:
```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
df["course_level"] = df["course_level"].map({
    "Beginner":1,
    "Intermediate":2,
    "Advanced":3
})

oe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

encoded = oe.fit_transform(df[['country_region', 'device_type',
                               'education_background', 'course_category']])

encoded_df = pd.DataFrame(encoded, columns=oe.get_feature_names_out())

df = pd.concat([df.drop(columns=['country_region', 'device_type',
                               'education_background', 'course_category']), encoded_df],axis=1)
```

[12]:
```
X = df.drop(["completion_status", "final_score"], axis=1)
y_class = df["completion_status"]
y_reg = df["final_score"]
```

[13]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y_class, test_size=0.2, random_state=32, stratify=y_class
)
```

[14]:
```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
   X, y_reg, test_size=0.2, random_state=32
)
```

[15]:
```
print("Classification shape:", X_train.shape, X_test.shape)
print("Regression shape:", X_train_reg.shape, X_test_reg.shape)
```

```
Classification shape: (4160, 28) (1040, 28)
Regression shape: (4160, 28) (1040, 28)
```

[16]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train_scale = scaler.fit_transform(X_train)
X_test_scale = scaler.transform(X_test)

X_train_reg_scale = scaler.fit_transform(X_train_reg)
X_test_reg_scale = scaler.transform(X_test_reg)
```

Part C: Bagging (Bootstrap Aggregating)

[17]:
```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(estimator=DecisionTreeClassifier(),
                            random_state=32,
                            n_estimators=10)
bag_clf.fit(X_train_scale, y_train)
y_pred_clf_bag = bag_clf.predict(X_test_scale)
```

[18]:
```
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix, recall_score
print("Accuracy_score:", accuracy_score(y_test, y_pred_clf_bag))
print("f1_score:", f1_score(y_test, y_pred_clf_bag))
print("Classification report:\n", classification_report(y_test, y_pred_clf_bag))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred_clf_bag))
print("recall_score:\n", recall_score(y_test, y_pred_clf_bag))
```

```
Accuracy_score: 0.6951923076923077
f1_score: 0.5425685425685426
Classification report:
              precision    recall  f1-score   support

           0       0.73      0.82      0.77       650
           1       0.62      0.48      0.54       390

    accuracy                           0.70      1040
   macro avg       0.67      0.65      0.66      1040
```

[23]:
```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

dt_clf = DecisionTreeClassifier(random_state=32)

dt_clf.fit(X_train_scale, y_train)

y_pred_dt = dt_clf.predict(X_test_scale)
y_pred_dt
```

[23]:
```
array([1, 0, 1, ..., 0, 1, 0], shape=(1040,))
```

Part D: Boosting Algorithms

[24]:
```
from sklearn.ensemble import AdaBoostClassifier
ada_clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=10),
    n_estimators=10,
    random_state=32
)

ada_clf.fit(X_train_scale, y_train)
y_pred_clf_ada = ada_clf.predict(X_test_scale)
```

[25]:
```
print("Accuracy:", accuracy_score(y_test, y_pred_clf_ada))
print("Classification_report:\n", classification_report(y_test, y_pred_clf_ada))
print("confusion_metrix:\n", confusion_matrix(y_test, y_pred_clf_ada))
print("f1_score:", f1_score(y_test, y_pred_clf_ada))
```

```
Accuracy: 0.6548076923076923
Classification_report:
              precision    recall  f1-score   support

           0       0.72      0.73      0.73       650
           1       0.54      0.53      0.53       390

    accuracy                           0.65      1040
   macro avg       0.63      0.63      0.63      1040
weighted avg       0.65      0.65      0.65      1040

confusion_metrix:
 [[475 175]
 [184 206]]
```

[28]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import AdaBoostRegressor

ada_reg = AdaBoostRegressor(estimator=DecisionTreeRegressor(max_depth=3),
                            random_state=32,
                            n_estimators=10)

ada_reg.fit(X_train_reg_scale, y_train_reg)

y_pred_reg_ada = ada_reg.predict(X_test_scale)
y_pred_reg_ada
```

[28]:
```
array([75.20294118, 67.27862903, 75.44029952, ..., 67.34563492,
       81.18049922, 68.58149847], shape=(1040,))
```

[29]:
```
print("mean_squared_error:", mean_squared_error(y_test, y_pred_reg_ada))
print("mean_absolute_error:", mean_absolute_error(y_test, y_pred_reg_ada))
print("r2_score:", r2_score(y_test,y_pred_reg_ada))
```

```
mean_squared_error: 5570.382828227976
mean_absolute_error: 74.32631856613206
r2_score: -23765.966733772697
```

Gradient Boosting

[30]:
```
from sklearn.ensemble import GradientBoostingClassifier
gb_clf = GradientBoostingClassifier(n_estimators=100,
                                    random_state=32,
                                    max_depth=3,
                                    learning_rate=0.1)

gb_clf.fit(X_train_scale, y_train)
pred_gb_clf = gb_clf.predict(X_test_scale)
pred_gb_clf
```

[30]:
```
array([0, 0, 0, ..., 0, 1, 0], shape=(1040,))
```

[31]:
```
print("Accuracy:", accuracy_score(y_test, pred_gb_clf))
print("f1_score:", f1_score(y_test, pred_gb_clf))
print("Classification_report:\n", classification_report(y_test, pred_gb_clf))
print("Confusion_matrix:\n", confusion_matrix(y_test, pred_gb_clf))
```

```
Accuracy: 0.7086538461538462
f1_score: 0.5785813630041725
Classification_report:
              precision    recall  f1-score   support

           0       0.74      0.81      0.78       650
           1       0.63      0.53      0.58       390

    accuracy                           0.71      1040
   macro avg       0.69      0.67      0.68      1040
weighted avg       0.70      0.71      0.70      1040

Confusion_matrix:
 [[529 121]
 [182 208]]
```

[32]:
```
from sklearn.ensemble import GradientBoostingRegressor
gb_reg = GradientBoostingRegressor(
    random_state=32,
    n_estimators=100,
    max_depth=3,
    learning_rate=0.1
)

gb_reg.fit(X_test_reg_scale, y_test_reg)

y_pred_gb_reg = gb_reg.predict(X_test_scale)
```

[33]:
```
print("Mean absolute error:", mean_absolute_error(y_test, y_pred_gb_reg))
print("Mean squared error:", mean_squared_error(y_test, y_pred_gb_reg))
print("r2_score:", r2_score(y_test, y_pred_gb_reg))
```

```
Mean absolute error: 74.5114969176868
Mean squared error: 5627.84659646656
r2_score: -24011.145478257324
```

[34]:
```
for lr in[0.01, 0.1,0.5]:
    for n in[50, 100, 150]:
        model = GradientBoostingClassifier(
            random_state=32, learning_rate=lr, n_estimators=n
        )
        model.fit(X_train_scale, y_train)
        pred = model.predict(X_test_scale)

        print("learning rate:", lr)
        print("Estimator:", n)
        print("Accuracy:", accuracy_score(y_test, pred))
```

```
learning rate: 0.01
Estimator: 50
Accuracy: 0.6451923076923077
learning rate: 0.01
Estimator: 100
Accuracy: 0.7009615384615384
learning rate: 0.01
Estimator: 150
Accuracy: 0.7067307692307693
learning rate: 0.1
Estimator: 50
Accuracy: 0.7163461538461539
learning rate: 0.1
Estimator: 100
Accuracy: 0.7086538461538462
learning rate: 0.1
Estimator: 150
Accuracy: 0.7115384615384616
learning rate: 0.5
Estimator: 50
Accuracy: 0.7
learning rate: 0.5
Estimator: 100
Accuracy: 0.6971153846153846
learning rate: 0.5
Estimator: 150
Accuracy: 0.6951923076923077
```

Light Gradient Boosting

[35]:
```
from lightgbm import LGBMClassifier
lgbm_clf = LGBMClassifier(
    random_state=32,
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    verbosity=-1

)

lgbm_clf.fit(X_train_scale, y_train)
y_pred_lgbm = lgbm_clf.predict(X_test_scale)
y_pred_lgbm
```

[35]:
```
array([0, 0, 0, ..., 0, 1, 0], shape=(1040,))
```

[36]:
```
print("Accuracy:", accuracy_score(y_test, y_pred_lgbm))
print("Classification report:\n", classification_report(y_test, y_pred_lgbm))
print("Confusion matrix\n:", confusion_matrix(y_test, y_pred_lgbm))
print("f1_score:", f1_score(y_test, y_pred_lgbm))
```

```
Accuracy: 0.7096153846153846
Classification report:
               precision    recall  f1-score   support

           0       0.75      0.81      0.78       650
           1       0.63      0.54      0.58       390

    accuracy                           0.71      1040
   macro avg       0.69      0.67      0.68      1040
weighted avg       0.70      0.71      0.70      1040

Confusion matrix
: [[529 121]
 [181 209]]
f1_score: 0.5805555555555556
```

XGBoost

[39]:
```
from xgboost import XGBClassifier

xgb_clf = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=32,
    eval_metric='logloss'
)

xgb_clf.fit(X_train_scale, y_train)

y_pred_xgb = xgb_clf.predict(X_test_scale)
```

[40]:
```
print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("Classification report:\n", classification_report(y_test, y_pred_xgb))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred_xgb))
print("f1 score:", f1_score(y_test, y_pred_xgb))
```

```
Accuracy: 0.7125
Classification report:
              precision    recall  f1-score   support

           0       0.75      0.81      0.78       650
           1       0.63      0.55      0.59       390

    accuracy                           0.71      1040
   macro avg       0.69      0.68      0.68      1040
weighted avg       0.71      0.71      0.71      1040

Confusion matrix:
 [[525 125]
 [174 216]]
f1 score: 0.5909712722298222
```

Part E: Voting & Stacking Ensembles

[41]:
```
from sklearn.ensemble import VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

voting_clf = VotingClassifier(
    estimators=[
        ('dt', DecisionTreeClassifier(random_state=32)),
        ('nb', GaussianNB()),
        ('knn', KNeighborsClassifier(n_neighbors=5)),
        ('lr', LogisticRegression(max_iter=1000))
    ],
    voting='hard'
)

voting_clf.fit(X_train_scale, y_train)

y_pred_voting = voting_clf.predict(X_test_scale)
```

[42]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Accuracy:", accuracy_score(y_test, y_pred_voting))
print("\nClassification Report:\n", classification_report(y_test, y_pred_voting))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_voting))
```

```
Accuracy: 0.698076923076923

Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.88      0.78       650
           1       0.66      0.40      0.50       390

    accuracy                           0.70      1040
   macro avg       0.68      0.64      0.64      1040
weighted avg       0.69      0.70      0.68      1040
```

[45]:
```
from sklearn.ensemble import StackingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.linear_model import LogisticRegression

stack_clf = StackingClassifier(
    estimators=[
        ('dt', DecisionTreeClassifier(random_state=32)),
        ('nb', GaussianNB()),
        ('knn', KNeighborsClassifier(n_neighbors=5))
    ],
    final_estimator=LogisticRegression(),
    cv=5
)

stack_clf.fit(X_train_scale, y_train)

y_pred_stack = stack_clf.predict(X_test_scale)
```

[46]:
```
print("Accuracy:", accuracy_score(y_test, y_pred_stack))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_stack))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_stack))
```

```
Accuracy: 0.7028846153846153

Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.85      0.78       650
           1       0.65      0.46      0.54       390

    accuracy                           0.70      1040
   macro avg       0.68      0.65      0.66      1040
weighted avg       0.69      0.70      0.69      1040
```

[49]:
```
from sklearn.ensemble import StackingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import LinearRegression

stack_reg = StackingRegressor(
    estimators=[
        ('dt', DecisionTreeRegressor(random_state=32)),
        ('knn', KNeighborsRegressor(n_neighbors=5))
    ],
    final_estimator=LinearRegression(),
    cv=5
)

stack_reg.fit(X_train_reg_scale, y_train_reg)

y_pred_stack_reg = stack_reg.predict(X_test_reg_scale)
```

[50]:
```
print("MAE:", mean_absolute_error(y_test_reg, y_pred_stack_reg))
print("MSE:", mean_squared_error(y_test_reg, y_pred_stack_reg))
print("R2 Score:", r2_score(y_test_reg, y_pred_stack_reg))
```

```
MAE: 9.205966350488174
MSE: 125.78206527586354
R2 Score: 0.28442194496255613
```

[53]:
```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

models = {
    "Bagging": bag_clf,
    "AdaBoost": ada_clf,
    "Gradient Boosting": gb_clf,
    "LightGBM": lgbm_clf,
    "XGBoost": xgb_clf,
    "Voting": voting_clf,
    "Stacking": stack_clf
}

for name, model in models.items():
    pred = model.predict(X_test_scale)

    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred))
    print("Recall:", recall_score(y_test, pred))
    print("F1 Score:", f1_score(y_test, pred))

    if name == "Voting":
        print("ROC-AUC: Not Available (Hard Voting)")
    else:
        prob = model.predict_proba(X_test_scale)[:, 1]
        print("ROC-AUC:", roc_auc_score(y_test, prob))
```

```
 Bagging
Accuracy: 0.6951923076923077
Precision: 0.6204620462046204
Recall: 0.48205128205128206
F1 Score: 0.5425685425685426
ROC-AUC: 0.7259704142011835

 AdaBoost
Accuracy: 0.6548076923076923
Precision: 0.5406824146981627
Recall: 0.5282051282051282
F1 Score: 0.5343709468223087
ROC-AUC: 0.6824753451676528

 Gradient Boosting
Accuracy: 0.7086538461538462
Precision: 0.6322188449848024
Recall: 0.5333333333333333
F1 Score: 0.5785813630041725
ROC-AUC: 0.7653767258382643

 LightGBM
Accuracy: 0.7096153846153846
Precision: 0.6333333333333333
Recall: 0.5358974358974359
F1 Score: 0.5805555555555556
ROC-AUC: 0.7666074950690335

 XGBoost
Accuracy: 0.7125
Precision: 0.6334310850439883
Recall: 0.5538461538461539
F1 Score: 0.5909712722298222
ROC-AUC: 0.7665009861932939

 Voting
Accuracy: 0.698076923076923
Precision: 0.6610169491525424
Recall: 0.4
F1 Score: 0.4984025559105431
ROC-AUC: Not Available (Hard Voting)

 Stacking
Accuracy: 0.7028846153846153
Precision: 0.6462093862815884
```

[54]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

models = {
    "AdaBoost": y_pred_reg_ada,
    "Gradient Boosting": y_pred_gb_reg,
    "LightGBM": y_pred_lgbm_reg,
    "Stacking": y_pred_stack_reg
}

for name, pred in models.items():
    print("\n", name)
    print("MAE:", mean_absolute_error(y_test_reg, pred))
    print("RMSE:", np.sqrt(mean_squared_error(y_test_reg, pred)))
    print("R2 Score:", r2_score(y_test_reg, pred))
```

```
 AdaBoost
MAE: 12.013156260088861
RMSE: 14.652792332208117
R2 Score: -0.2214595269772741

 Gradient Boosting
MAE: 12.747087131421653
RMSE: 15.653651289502989
R2 Score: -0.3940219174890742

 LightGBM
MAE: 7.884443833527114
RMSE: 9.822218783618414
R2 Score: 0.4511451589964953

 Stacking
MAE: 9.205966350488174
RMSE: 11.215260374858158
R2 Score: 0.28442194496255613
```

[55]:
```
# Classification comparison
classification = pd.DataFrame({
    "Model": ["Bagging", "AdaBoost", "Gradient Boosting", "LightGBM",
              "XGBoost", "Voting", "Stacking"],
    "Accuracy": [0.6952, 0.6548, 0.7087, 0.7096, 0.7125, 0.6942, 0.7029],
    "F1 Score": [0.5426, 0.5344, 0.5786, 0.5806, 0.5910, 0.5000, 0.5400]
})

print("Classification Model Comparison")
display(classification)

print("Best Classification Model: XGBoost")


# Regression comparison
regression = pd.DataFrame({
    "Model": ["AdaBoost", "Gradient Boosting", "LightGBM", "Stacking"],
    "MAE": [74.3263, 74.5115, 74.3821, 9.2060],
    "RMSE": [74.6350, 75.0190, 74.9895, 11.2153],
    "R2 Score": [-23765.9667, -24011.1455, -23992.2484, 0.2844]
})

print("\nRegression Model Comparison")
display(regression)

print("Best Regression Model: Stacking Regressor")
```

Classification Model Comparison

	Model	Accuracy	F1 Score
0	Bagging	0.6952	0.5426
1	AdaBoost	0.6548	0.5344
2	Gradient Boosting	0.7087	0.5786
3	LightGBM	0.7096	0.5806
4	XGBoost	0.7125	0.5910
5	Voting	0.6942	0.5000
6	Stacking	0.7029	0.5400

Best Classification Model: XGBoost

Regression Model Comparison

	Model	MAE	RMSE	R2 Score
0	AdaBoost	74.3263	74.6350	-23765.9667
1	Gradient Boosting	74.5115	75.0190	-24011.1455
2	LightGBM	74.3821	74.9895	-23992.2484
3	Stacking	9.2060	11.2153	0.2844

Best Regression Model: Stacking Regressor